## 3.2. Replacement policies

→ doar pentru set-associative și full-associative caches

a) FIFO → nu se prea folosește
→ not compliant with the locality principle

b) Random : când s e mic

„pseudo random"

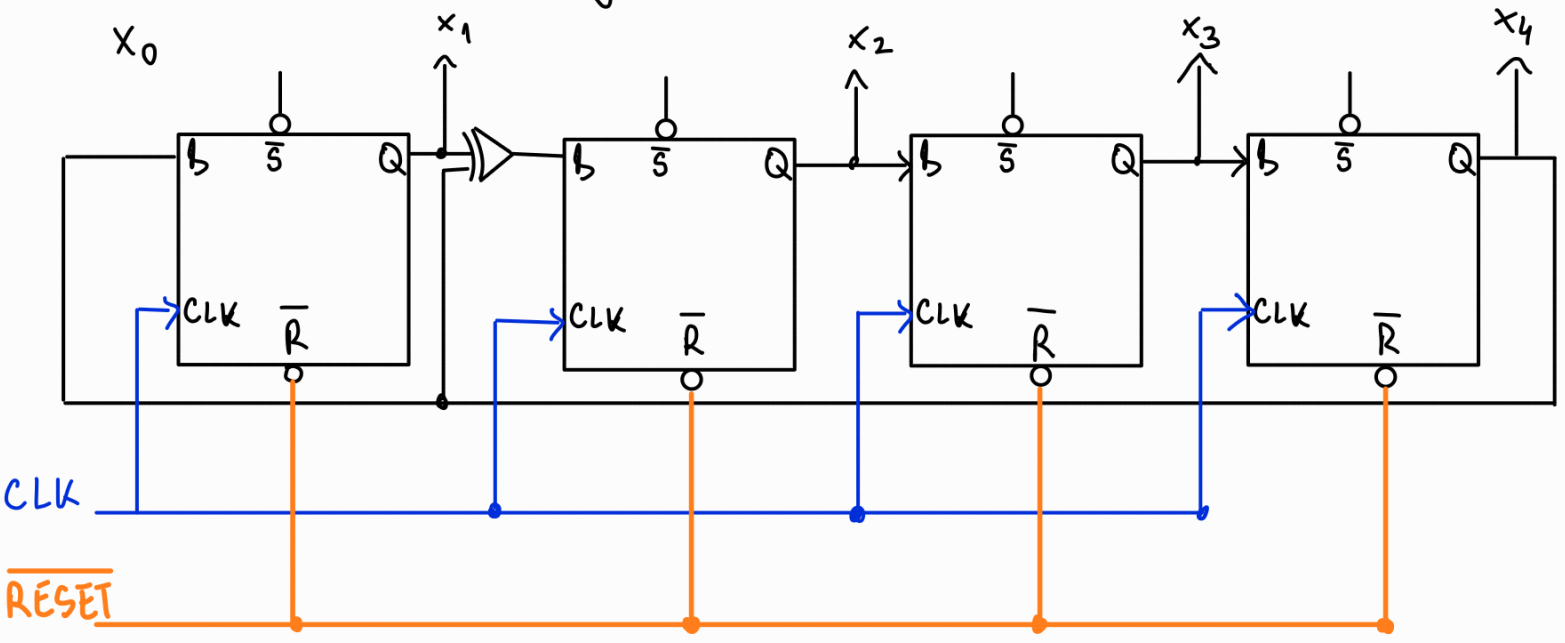
→ Modul de generare random al numerelor : prin polinom indivizibil

Galois Field Polynomials $G(x) = x^4 + x + 1$

$\deg(G(x)) = 4$

$\text{period} = 2^{\deg(G(x))} - 1$

Linear Feedback Shift Register (LSFR)




perioada = $2^4 - 1 = 15$ clock-uri

→ not compliant with the locality principle

RESET

| | $b_0$ | $b_1$ | $b_2$ | $b_3$ | Decimal |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 | 0 | 2 |
| 3 | 0 | 0 | 1 | 0 | 4 |
| 4 | 0 | 0 | 0 | 1 | 8 |
| 5 | 1 | 1 | 0 | 0 | 3 |
| 6 | 0 | 1 | 1 | 0 | 6 |
| 7 | 0 | 0 | 1 | 1 | 12 |
| 8 | 1 | 1 | 0 | 1 | 11 |
| 9 | 1 | 0 | 1 | 0 | 5 |
| 10 | 0 | 1 | 0 | 1 | 10 |
| 11 | 1 | 1 | 1 | 0 | 7 |
| 12 | 0 | 1 | 1 | 1 | 14 |
| 13 | 1 | 1 | 1 | 1 | 15 |
| 14 | 1 | 0 | 1 | 1 | 13 |
| 15 | 1 | 0 | 0 | 1 | 9 |
| 16 | 1 | 0 | 0 | 0 | 1 |
| | 0 | 1 | 0 | 0 | 2 |
| | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |

## c) Least Recently Used (LRU)

→ age registers

→ each block in a set has an age

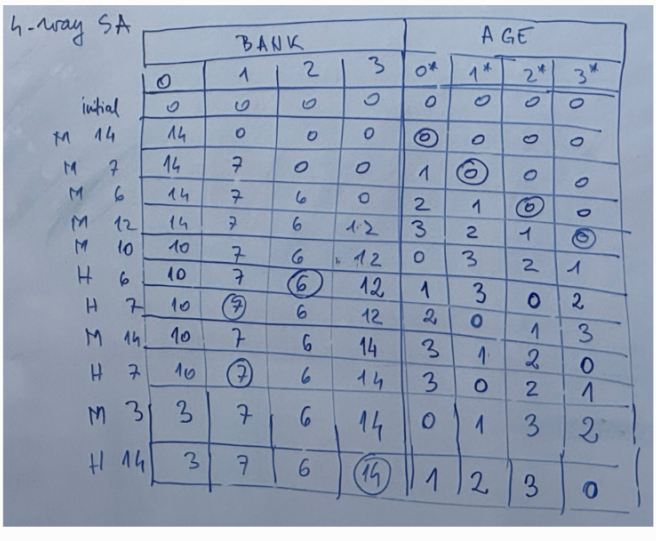
→ highest age block is replaced

→ no. of age bits = $\log_2$ (set associativity)

Exemplu

Pp. că avem un set asociativ pe 4 căi cu adrese în zecimal și p.p. că toate adresele se mapează la același index

14, 7, 6, 12, 10, 6, 7, 14, 7, 3, 14 (assumed to have the same index)




MISS - replace the LRU
- put the age 0 for replaced bank
- increment the ages of other banks

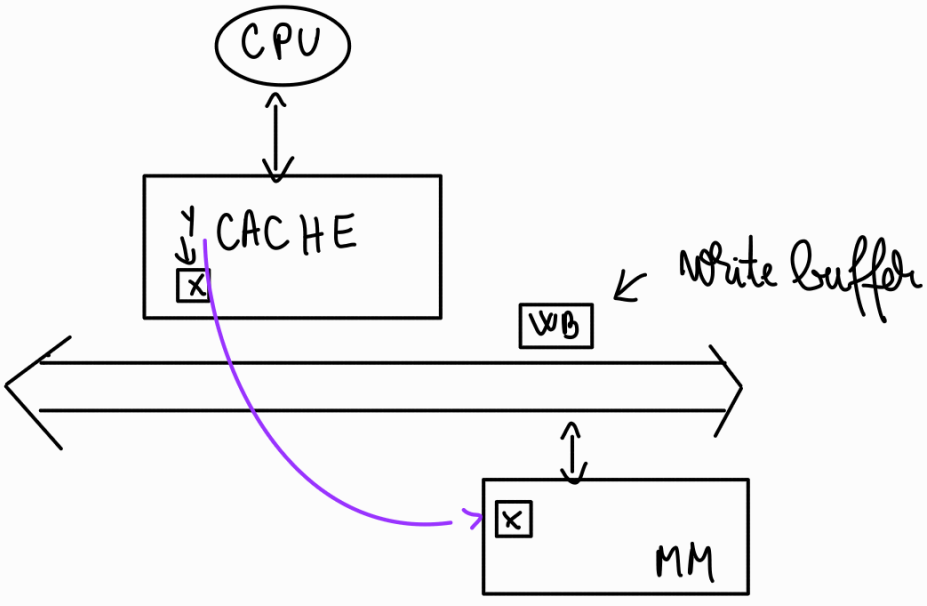
HIT - put the age 0 for the hit block
- increment the ages that were lower than the age of the hit block

## 3.3. Write policies

– 10% of all accesses are writes ⇒ cache / memory coherence ⟶ update

1. Write Through

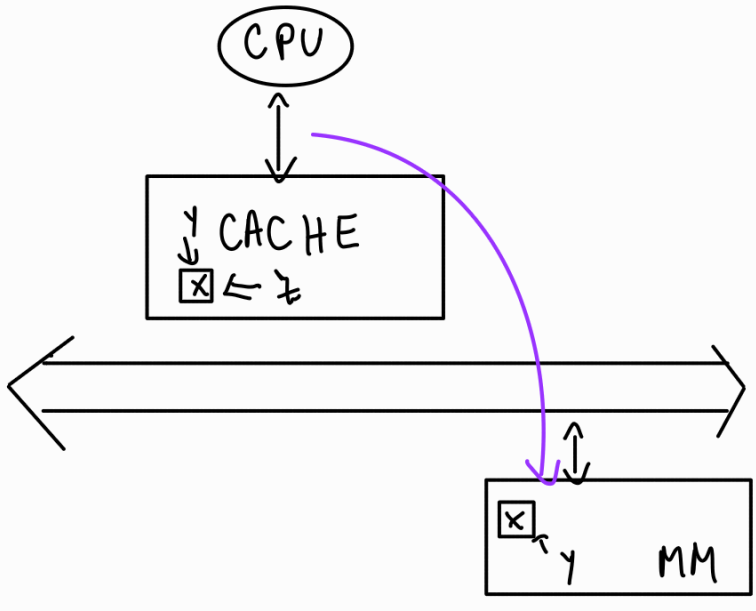
– write buffer




⟶ când se scrie în cache, se înlocuiește direct și în MM

⟶ se pierde mai mult timp

2. Write back – update upon replacement




⟶ dirty bit: dacă blocurile sunt dirty, bitul e 1 și doar acele blocuri trebuie înlocuite

What happens in case of a write miss?

( What happens in case of a read miss ⟶ allways allocate)

⇒ allocate

α) Write Allocate

β) Write No Allocate

Pairings { WT + β ; WB + α }

## 3.4. Cache performance

Metrics
- miss rate (MR) ; miss = când CPU nu găsește informația căutată în Cache
  Miss Rate = 1 − Hit Rate
- AMAT ( Average Memory Access Time)
  AMAT = $t_{ac}$ + Miss Rate · Miss Penalty (Time)
- CPU time
- $t_{ac}$ = timp de acces la cache
  (default $t_{ac}$ = 1 CC → clock cycles ; în s × CCT)

- Miss causes
  - → compulsory ( cold-start misses) : miss-uri apărute când resetezi sistemul și cache-ul e gol
    soluție: bigger block size
  - → conflict - multiple blocks map at the same indexe
    soluție: higher SA (s)
  - → capacity - memorie cache mică
    soluție: bigger cache size

$$AMAT = t_{ac} + \text{Miss Rate} \cdot \text{Miss Penalty} = \quad (\text{Miss Penalty} \rightarrow \text{Time})$$

$$= (1 + \text{Miss Rate} \cdot \text{Miss Penalty}) \cdot CCT \quad (CCT \rightarrow \text{Clock Cycle Time})$$

(Miss Penalty $\rightarrow$ CC)

$$CPU_{time_{ideal}} = ic \cdot CPi \cdot CCT \rightarrow \text{for a perfect cache}$$

$$CPU_{time} = ic \cdot (CPi_{ideal} + \text{Memory accesses per instr} \cdot \text{Miss Rate} \cdot \text{Miss Penalty}) \cdot CCT \quad (\text{Miss Penalty} \rightarrow CC)$$

miss penalty -> timpul care îl ia căutarea în MM

$t_{ac}$ $\longrightarrow$ CPU $\leftrightarrow$ CACHE, timp hit